Introduction to APIs: Wikipedia

Week 10 · APIs module · Textbook Chapter 10

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 19, 21 & 23, 2026

This week at a glance

We leave scraping behind and start asking servers for data *on purpose* — through a documented API.

Monday | Concepts & notebook intro

- What an API is, why it beats scraping, and why we learn on Wikipedia

Wednesday | Notebook lab

- Query the MediaWiki API: revisions, pageviews, links, raw requests

Friday | Textbook revisions |Proposal due

- Pull requests improving Chapter 10 — **and** your Final Project proposal

Companion notebook

`ch-10-wikipedia.ipynb`

Due Friday, Oct 23

Final Project proposal (5%) — a research question, your data source(s) and method, and an ethics & feasibility check.

1. Monday · Concepts

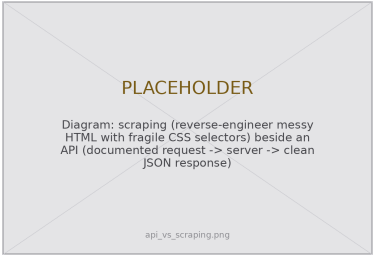
From scraping to APIs

For six weeks we **reverse-engineered** pages built for human eyes — inspecting HTML, guessing selectors, and cleaning up the mess.

This week begins Part III: **API-based data access**. Instead of taking data a site never meant to hand us, we ask for it the way the operator intends.

The skills you build here — reading documentation, constructing parameterized URLs, handling pagination, parsing nested JSON — transfer to *every* API in the rest of the course.

When an API exists, it is almost always the better tool.



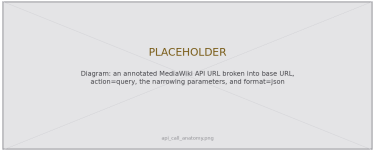
Ask, don't reverse-engineer.

What is an API?

An **API** (Application Programming Interface) is a **contract**. It specifies what you can request. It specifies which parameters you can send. It also specifies the format of the response.

- Clean **JSON**, not cluttered HTML
- **Stable endpoints**, not fragile CSS selectors
- **Documentation** tells you exactly what exists

The tradeoff: an API only exposes what its operator *chooses* to. You cannot request a field that the API does not include. You cannot exceed the rate limits. You cannot rely on the API forever. The "post-API age" later in this module lists many APIs that operators restricted, repriced, or shut down.



A request is a URL you assemble on purpose.

The case for Wikipedia

The **MediaWiki API** is a near-perfect place to learn:

- Free, with no authentication beyond a **User-Agent** header
- Comprehensive, well-maintained documentation
- Data rich enough for real research — content, networks, events

Our case study: the `Elizabeth_II` article. Its editing history has dramatic spikes around major events — ideal for seeing temporal patterns in API data.



Every edit, ever, is queryable.

Public interest

Wikipedia is the counter-example to enclosure: an **open API**, an **open license**, and public database **dumps**. Nothing you learn here sits at the mercy of a pricing change.

The anatomy of a request — and what the notebook covers

Every MediaWiki call is four parts:

1. a **base URL** — `.../w/api.php`
2. an `action` — what you want to do
3. **parameters** that narrow the request
4. a `format` — `json`

The response is nested JSON you navigate **one level at a time**: `data["query"]["pages"][page_id]`. Internalize that shape once and every endpoint feels familiar.

This week's notebook

- Revisions + pagination
- Pageviews (a second API)
- Links, categories, languages
- Raw requests from the docs

Connecting to Ch. 2 (Ethics)

Identify yourself: the Wikimedia **User-Agent policy** asks every client for a descriptive agent with contact info. Anonymous traffic gets throttled.

Your Final Project proposal, due Friday

Three parts, worth 5% of your grade:

1. **A research question.** What the data could answer, and who would care about the answer. A question, not a topic: not "Wikipedia," but "Which events send readers to articles about wildfires?"
2. **Data source(s) and method.** The site, API, or documents, and how you'll get them: an API, scraping, PDFs, or a mix. Name the endpoint, page, or file.
3. **Ethics and feasibility.** The terms of service, `robots.txt`, and any personal data in it — and proof that you can get **one record**: paste the request and what came back.

This week makes step 3 easy

If your source has an API, Wednesday's notebook is your feasibility check: one request, one record, the status code, and the fields you'll use.

Stuck for an idea?

Week 9's Friday slides grow every chapter's *Additional Exercises* into project questions.

Collecting over time?

Start the collector now, and say so in the proposal.

2. Wednesday · Notebook Lab

The four parts, then navigate the JSON

Assemble the request as a dictionary of parameters; `requests` builds the URL for you:

```
import requests
URL = "https://en.wikipedia.org/w/api.php"
HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}

params = {"action": "query", # what to do
          "titles": "Elizabeth_II", # about this page
          "prop": "info", # what we want back
          "format": "json"} # how we want it
data = requests.get(URL, params=params, headers=HEADERS).json()
```

Then walk the nested response one key at a time — page IDs are the keys:

```
pages = data["query"]["pages"]
page_id = list(pages.keys())[0]
print(pages[page_id]["title"], pages[page_id]["pageid"])
```

Revisions — and the pagination loop

```python
def get_revisions(title, headers=HEADERS):
    params = {"action": "query", "titles": title,
        "prop": "revisions", "rvlimit": "max",
        "rvprop": "ids|timestamp|user|size|comment",
        "format": "json"}
    revs = []
    while True: # the pagination loop
        data = requests.get(URL, params=params,
                            headers=headers).json()
        page = list(data["query"]["pages"].values())[0]
        revs.extend(page.get("revisions", []))
        if "continue" not in data: # last batch -> stop
            break
        params.update(data["continue"]) # else: next batch
    return pd.DataFrame(revs)
```

The single most common API bug: **forgetting to paginate**. The server returns results in batches (here, capped at 500) and hands back a `continue` token when more remain.

Skip the loop and you silently keep a *fraction* of the data.

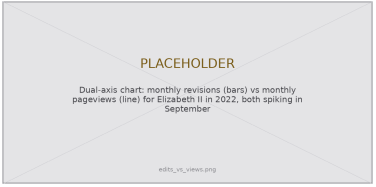
Two habits: `raise_for_status()` to fail loudly, and `maxlag=5` so bulk jobs yield to human readers.

Same pattern, more endpoints

Pageviews live on a **different** API (REST) — parameters are baked into the URL *path*, and there is no continuation:

```
def get_pageviews(title, start, end, headers=HEADERS):
    url = ("https://wikimedia.org/api/rest_v1/metrics"
           "/pageviews/per-article/en.wikipedia"
           f"/all-access/all-agents/{title}"
           f"/daily/{start}/{end}")
    r = requests.get(url, headers=headers)
    r.raise_for_status()
    return pd.DataFrame(r.json()["items"])
```

Links, categories, and languages reuse the *same* Action-API loop — just swap `prop` and the matching `*limit` key (`links`, `categories`, `langlinks`).



Edits vs. views, 2022 — both spike in September.

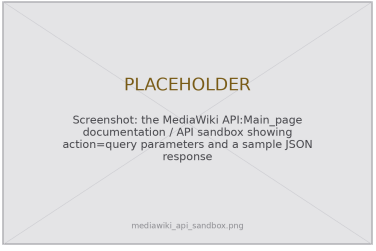
How many of Wikipedia's 300+ languages carry an article is a rough proxy for **global significance** — one that skews toward the Global North.

From documentation to working code

The transferable skill is writing a request from the docs alone — no wrapper. Read the parameters for an endpoint you have never used, build it, inspect the response:

```
params = {"action": "query",
          "list": "search", # a NEW endpoint, learned from the docs
          "srsearch": "web scraping",
          "srlimit": 10,
          "srprop": "wordcount|timestamp|snippet",
          "format": "json"}
data = requests.get(URL, params=params, headers=HEADERS).json()
for r in data["query"]["search"]:
    print(r["title"], "-", r["wordcount"], "words")
```

Libraries like `mwclient` and `pywikibot` automate continuation and throttling — but we write raw requests first so you know exactly what they do for you.



Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the chapter:

1. Compare the revision histories of two related articles. Report the unique editors, the edits for each editor, and the monthly activity.
2. Get the daily pageviews of five articles. Report if the spikes occur together or separately.
3. Find the outgoing wikilinks of three related articles. Report the links that they share.
4. Read the API documentation. Write a request for a new endpoint such as `extlinks` or `recentchanges`.
5. Find the 20 most frequent editors of an article. Then find the other articles that they edit.
6. **5617**: reproduce one measure from Mesgari et al. (2015). For example, calculate a Gini coefficient of the edits for each editor

Show-and-Tell

Bring one of these to class:

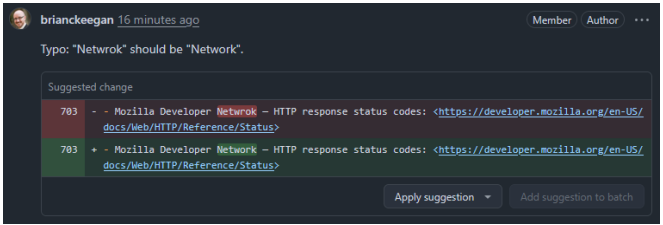
- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

Scraping asks: *how is this page built?*
An API asks: *what does the documentation promise?*

Learn one API by hand,
and you have learned them all.

3. Friday · Textbook Revisions

Improving Chapter 10 together



A review suggestion on the instructor's demonstration pull request: the author can apply it with one click.

Today we make the book better. Each of you opens a **pull request** against `ch-10-wikipedia.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

Also due today — Fri, Oct 23

Your **Final Project proposal** (5%): a research question, your data source(s) and method, and an ethics & feasibility check — with one record you actually retrieved.

Before you submit the proposal

- ☐ Is the question one the data can answer — and would anyone outside this class care?
- ☐ Does it name the exact source: an endpoint, a site's pages, or a set of PDFs?
- ☐ Did you retrieve **one real record**, and paste the request and the response?
- ☐ Did you read the terms of service and `robots.txt`? Does the data include people's names or other personal information, and how will you handle it?
- ☐ If the data changes over time, is your collector already running?
- ☐ Working in a pair? Ask the instructor before you submit.

What comes next

Presentation, about 5 minutes (10%), in the final week, Nov. 30–Dec. 4. Repository and a 2,000–3,000-word write-up (25%), due Monday, December 7.

Late work

Loses 1% of its maximum value for each day after the deadline, unless you have an approved excuse.

A revision menu for Chapter 10

High-value targets — pick one and scope it to a single PR:

- **Close a gap in "Common Issues to Debug."** Add a worked **missing-page** check (a bad title returns page ID `-1`), or a `urllib.parse.quote()` example for titles with parentheses / accents.
- **Show the etiquette, don't just name it.** The chapter mentions `maxlag` but never demonstrates it — add a small retry-on-`maxlag` example for bulk jobs.
- **Add a figure.** A diagram of the request anatomy, or of the request → `continue` → request pagination loop, would anchor the two hardest ideas.
- **Strengthen an exercise.** Exercises 1–5 have no expected output or starter cell — add a rubric or a self-checking assertion.
- **Deepen "Social History."** Turn the gender-gap discussion into a reproducible measurement (tie it to WikiProject Women in Red) using this chapter's helper functions.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Government APIs —** the Census, the Federal Reserve (FRED), and other public data services.

Key takeaways

1. An API is a **documented contract**: structured, intentional access — clean JSON and stable endpoints, no reverse-engineering.
2. The skill transfers: **read the docs** → **build a parameterized request** → **parse nested JSON** one level at a time.
3. Always **paginate**: follow the `continue` token in a loop, or you silently keep only the first batch.
4. Be a good client: a descriptive **User-Agent**, serial requests, and `maxlag` for bulk jobs (the ethics of Ch. 2, applied).
5. Openness is a *choice*, not a guarantee of completeness — Wikipedia's data still inherits its volunteer community's biases.